

NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Applied Sciences · Department of Computer Science

Solution Book 2

SCS 1101, 2025 paper

SCS 1101 Introduction to Computer Science and Programming

Past-paper solutions with marks allocated, algorithms, textbook flowcharts, exam-ready code and a civil & water engineering lens

5 questions · 25 marks each · cover page not supplied (assume any 4 of 5)

Book 2 of 8 · Prepared October 2026

Index — tap a tab or a line to jump

Every chapter starts with the coloured tab bar below; tap any tab to jump straight to that question. The page footer has a link back to this index. In Word, the Navigation Pane shows the same chapters.

Guide **How to use this book**

Glance **At a glance**

Q1 **Question 1 [25]**

- ▸ 1(a) Five ways computer viruses spread [10]
- ▸ 1(b) Program: ATM with deposit, withdraw and balance [15]

Q2 **Question 2 [25]**

- ▸ 2(a) Functions of computer components [2 each = 6]
- ▸ 2(b) The topology in Figure 1 [4]
- ▸ 2(c) Program: analyse 30 exam scores [15]

Q3 **Question 3 [25]**

- ▸ 3(a) Four types of errors, with examples [2 each = 8]
- ▸ 3(b) Local and global variables, with a program [7]
- ▸ 3(c) Compare and contrast traditional filing and cloud storage [10]

Q4 **Question 4 [25]**

- ▸ 4(a) Five benefits of using the internet [10]
- ▸ 4(b) Program: PAYE tax from salary [15]

Q5 **Question 5 [25]**

- ▸ 5(a) IaaS and PaaS, with examples [4 each = 8]
- ▸ 5(b) Five functions of an operating system [5]
- ▸ 5(c) Program: recursive Fibonacci series [12]

Refs **Sources for this book**

Links **Source register**

How to use this book

This book is one of eight stand-alone solution books for NUST SCS 1101 past papers. It solves **SCS 1101, 2025 paper** question by question, with the marks the examiner allocates shown in square brackets, e.g. [2].

The six-block answer template

Every answer in this book (and in all eight books of the set) is built from the same six blocks, in the same order. Theory answers use blocks 1, 2, 5 and 6; programming answers use all six.

1. **Like you're 3** — one or two sentences with a toy, a bath or a water tap.
2. **Model answer** — the exam wording, with the mark for each point in brackets, e.g. [2]. Write it like this and you collect every mark.
3. **Algorithm + code** — numbered pseudocode first, then C code that follows the **H-D-I-P-O-R** recipe.
4. **Flowchart or structure chart** — textbook (ANSI) symbols. Programs built from functions also get a structure (hierarchy) chart.
5. **Engineering lens** — the same idea in a civil and water engineering setting, using the profession's vocabulary.
6. **Examiner's tip** — where students usually lose marks.

The H-D-I-P-O-R code recipe — "**H**appy **D**ucks **I**n **P**onds **O**nly **R**elax". Every exam program is these six steps, in this order:

```c
/* H - Header: what the program does, and the libraries */
#include <stdio.h>

int main(void)
{
    /* D - Declare: every variable, with a sensible type and a starting value */
    /* I - Input:   printf a prompt, then scanf with & */
    /* P - Process: the if-ladder, loop or formula */
    /* O - Output:  printf the result with units and decimals */
    /* R - Return:  */ return 0;
}
```

The recipe is worth marks by itself: examiners give separate marks for headers, declarations, input, processing and output. A half-finished program laid out this way still scores; a clever one-liner without structure often does not.

Algorithm style used in every book:

```
Step 1: Start
Step 2: Read <inputs>
Step 3: <process, one action per step>
Step 4: Display <outputs>
Step 5: Stop
```

Flowchart symbols (ANSI):

Symbol	Shape	Use	Colour in this book
Terminator	Rounded box (stadium)	Start / Stop	Green

Symbol	Shape	Use	Colour in this book
Input/Output	Parallelogram	Read or display	Blue
Process	Rectangle	Calculation or assignment	Amber
Decision	Diamond	Yes/no test, two exits	Red
Predefined process	Rectangle with double sides	Call to a function	Purple
Flow line	Arrow	Order of steps	Dark grey arrow

The smallest possible program, drawn with these symbols:

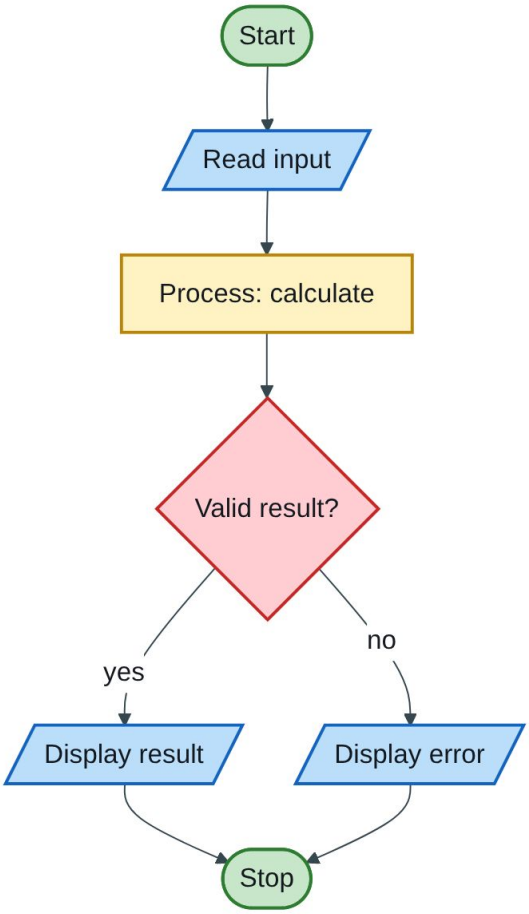


Figure 2.1 — Flowchart

The engineering lens. Programming questions get a civil and water engineering version where it fits, for example:

- tariff ladders become **increasing block tariffs** for water;
- array statistics become **rainfall** or **concrete cube strength** analysis;
- geometry programs become **cross-section, wetted perimeter and hydraulic radius** calculations.

Write the exam version in the exam; the engineering version is for understanding and for your later modules.

How the solutions were researched

Every fact a solution relies on is traced to a source in one of four tiers; each source has a short key (e.g. [MIT-2]) so answers can cite it without long links.

1. **Tier 1 — the course itself.** The SCS 1101 outline and the past papers decide *what* to answer and how marks are split.

2. **Tier 2 — university teaching material.** MIT OpenCourseWare and UCLA course pages for programming and algorithms; university histories for milestones.
3. **Tier 3 — standards and research.** NIST for cloud definitions; peer-reviewed and arXiv papers, including Wits research on cloud adoption in South Africa.
4. **Tier 4 — standard textbook knowledge.** Definitions every textbook agrees on (OS functions, topologies, printers) are written from the textbooks your outline lists, without a web citation.

A claim is only cited when a source was actually opened; anything from tier 4 stays general and uncited.

At a glance

Five questions of about 25 marks each (page 1 with the instructions was not provided; the 2024 rule was “any 4, no calculators”). Three questions end in a 12–15 mark program, so prepare the ATM, exam-scores, PAYE and Fibonacci programs.

Question	What it tests	Marks split	Pick?
Q1	How viruses spread; ATM menu program	10 theory + 15 program	Yes
Q2	ALU, address bus, register; star topology; 30 exam scores	10 theory + 15 program	Yes
Q3	Four error types; local vs global; filing vs cloud storage	25 theory with a short program	Good
Q4	Internet benefits; PAYE tax program	10 theory + 15 program	Yes
Q5	IaaS and PaaS; OS functions; recursive Fibonacci	13 theory + 12 program	Yes

Question 1 [25]

1(a) Five ways computer viruses spread [10]

Like you're 3: a virus is a germ for computers. It jumps from one computer to another through anything they share: a USB stick, an email, a download, a network cable.

Name the route [1] and explain how infection happens [1], five times:

1. **Infected email attachments** — opening a file (e.g. a fake invoice) runs the hidden virus code.
2. **Removable media** — USB flash disks and external drives carry the virus between machines, often through autorun files.
3. **Downloads from untrusted sites** — pirated software, cracked games and fake updates come bundled with malware.
4. **Malicious links and phishing** — a click on a link in an SMS, WhatsApp or email message leads to a site that installs malware.
5. **Networks and file sharing** — shared folders, peer-to-peer sharing and worms that exploit unpatched security holes spread across a LAN without anyone clicking.
6. **Infected documents (macros)** — Word or Excel files with malicious macros run when editing is enabled.

Engineering lens: in a water utility, malware brought in on a contractor's USB stick could reach the **SCADA** computers that run pumps and dosing. This is why control networks are separated from office networks (air-gapping or firewalls).

1(b) Program: ATM with deposit, withdraw and balance [15]

Like you're 3: a piggy bank that starts with $500. You can put money in, take money out (but never more than is inside), or peek at how much is there, again and again until you say "bye".

Algorithm:

```
Step 1: Start; balance = 500
Step 2: Display menu: 1 Deposit, 2 Withdraw, 3 Balance, 4 Exit
Step 3: Read choice
Step 4: If choice = 1: read amount; if amount > 0, balance = balance + amount
        If choice = 2: read amount; if amount > balance, display "Insufficient funds"
                       else balance = balance - amount
        If choice = 3: display balance
        If choice = 4: display goodbye
        Otherwise: display "Invalid option"
Step 5: If choice is not 4, go to Step 2
Step 6: Stop
```

Flowchart:

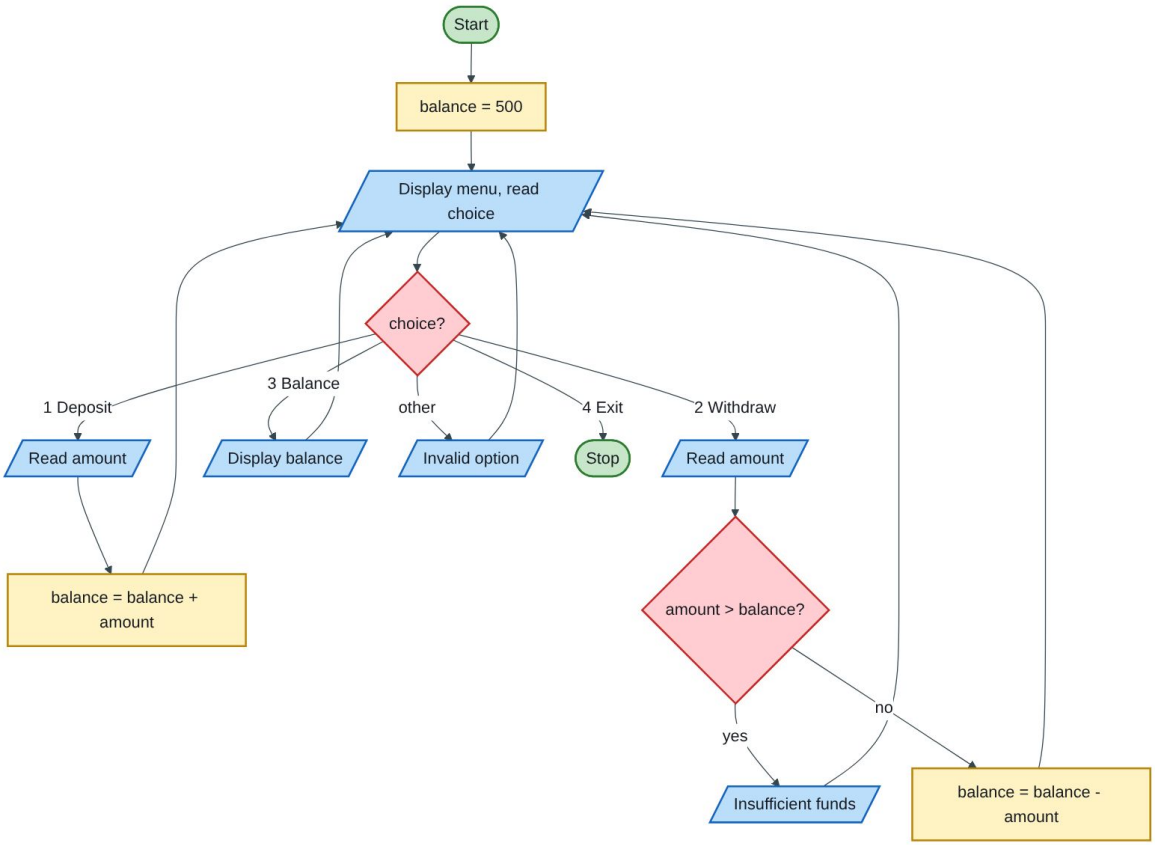


Figure 2.2 — Flowchart

Code (H-D-I-P-O-R) — the menu-loop pattern: `do { menu; switch } while (not exit);`

```c
/* H: ATM simulation - deposit, withdraw, check balance; opening balance $500 */
#include <stdio.h>

int main(void)
{
    double balance = 500.00, amount;            /* D */
    int choice;

    do {
        printf("\n====== ATM MENU ======\n");
        printf("1. Deposit\n2. Withdraw\n3. Check balance\n4. Exit\n");
        printf("Choose an option: ");
        scanf("%d", &choice);                   /* I */

        switch (choice) {                       /* P */
        case 1:                                 /* deposit */
            printf("Amount to deposit: $");
            scanf("%lf", &amount);
            if (amount > 0) {
                balance += amount;
                printf("Deposited $%.2f\n", amount);
            } else {
                printf("Invalid amount\n");
            }
            break;
        case 2:                                 /* withdraw, no overdraft */
            printf("Amount to withdraw: $");
            scanf("%lf", &amount);
            if (amount <= 0)
```

```c
                printf("Invalid amount\n");
            else if (amount > balance)
                printf("Insufficient funds. Balance is $%.2f\n", balance);
            else {
                balance -= amount;
                printf("Please take your $%.2f\n", amount);
            }
            break;
        case 3:                                     /* O */
            printf("Your balance is $%.2f\n", balance);
            break;
        case 4:
            printf("Thank you for banking with us.\n");
            break;
        default:
            printf("Invalid option, try again\n");
        }
    } while (choice != 4);

    return 0;                                       /* R */
}
```

Test run:

Action	Amount	Result	Balance after
Start	—	—	$500.00
Deposit	200	Accepted	$700.00
Withdraw	1000	Refused: insufficient funds	$700.00
Withdraw	300	Accepted	$400.00
Balance	—	Shows $400.00	$400.00
Exit	—	Goodbye	—

Marking guide [15]: opening balance and declarations [2]; menu display [2]; loop that repeats until exit [3]; deposit [2]; withdrawal with the overdraft check [3]; balance enquiry [1]; invalid input handled [1]; formatted money output [1].

Engineering lens: this is the logic inside a **prepaid water meter**. Buying a token is a *deposit* of credit; consumption is a *withdrawal*; and when credit reaches zero the meter's **valve shuts off**, the water version of "no overdraft". Utilities use prepaid metering to cut **non-revenue water** from unpaid bills.

Examiner's tip: check `amount > balance` **before** subtracting. Subtracting first and checking afterwards lets the balance go negative for a moment, which loses the overdraft mark.

Question 2 [25]

2(a) Functions of computer components [2 each = 6]

Like you're 3: the ALU is the calculator in the computer's brain; the address bus is the road sign saying which drawer to open; registers are the tiny notepads the brain keeps in its hand.

Component	Model answer [1 function + 1 detail]
Arithmetic and Logic Unit (ALU)	The part of the CPU that performs arithmetic operations (+, −, ×, ÷) [1] and logical operations and comparisons (AND, OR, NOT, greater than, equal to) on data supplied by the registers [1].
Address bus	A set of parallel lines that carries the **memory address** of the location the CPU wants to read from or write to [1]. It is one-way (CPU → memory), and its width sets how much memory can be addressed: 32 lines address 2^{32} locations (4 GB) [1].
Register	A very small, very fast storage location inside the CPU [1] that holds the data, instruction or address currently being used, e.g. the program counter (address of the next instruction) and the accumulator (results) [1].

2(b) The topology in Figure 1 [4]

(i) Identify it [1]: a **star topology**: every device has its own link to a central hub.

(ii) Two advantages and one disadvantage [3]:

- Advantage: a fault in one cable or device affects only that device; the rest keep working [1].
- Advantage: easy to install, manage and expand; adding a device needs only one new cable to the hub, and faults are easy to isolate [1].
- Disadvantage: the hub is a **single point of failure**; if it fails, the whole network stops. It also uses more cable than a bus [1].

Engineering lens: star is also how a utility's telemetry is usually laid out: each reservoir and pump-station outstation reports to one central control room (master station). If the master station fails, every site loses supervision, which is why control rooms have backup servers.

2(c) Program: analyse 30 exam scores [15]

Like you're 3: 30 children each get a sticker with their score. We look for the biggest sticker, the smallest sticker, share all the points out equally (average), and count the stickers under 50.

Algorithm:

```
Step 1: Start
Step 2: For i = 0 to 29: read score[i]  (reject values outside 0-100)
Step 3: highest = score[0], lowest = score[0], sum = 0, failed = 0
Step 4: For i = 0 to 29:
          sum = sum + score[i]
          if score[i] > highest: highest = score[i]
```

```
          if score[i] < lowest:  lowest = score[i]
          if score[i] < 50:      failed = failed + 1
Step 5: average = sum / 30
Step 6: Display highest, lowest, average, failed
Step 7: Stop
```

Flowchart (processing loop):

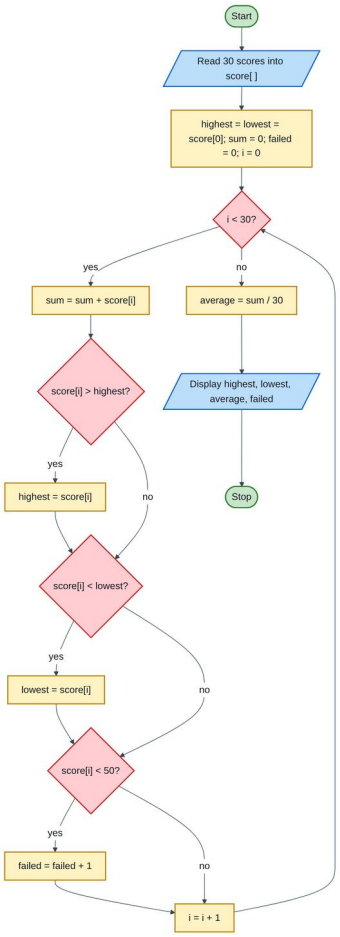


Figure 2.3 — Flowchart

Code (H-D-I-P-O-R):

```c
/* H: analyse 30 exam scores - highest, lowest, average, number failed */
#include <stdio.h>
#define N 30
#define PASS 50

int main(void)
{
    int score[N];                                   /* D */
    int i, highest, lowest, failed = 0;
    double sum = 0.0, average;

    for (i = 0; i < N; i++) {                       /* I: read and validate */
        do {
            printf("Score for student %d (0-100): ", i + 1);
            scanf("%d", &score[i]);
        } while (score[i] < 0 || score[i] > 100);
    }
```

```
    highest = lowest = score[0];                    /* P: start from the first score
*/
    for (i = 0; i < N; i++) {
        sum += score[i];
        if (score[i] > highest) highest = score[i];
        if (score[i] < lowest)  lowest  = score[i];
        if (score[i] < PASS)    failed++;
    }
    average = sum / N;

    printf("\nHighest score : %d\n", highest);      /* O */
    printf("Lowest score  : %d\n", lowest);
    printf("Average score : %.2f\n", average);
    printf("Failed (< 50) : %d student(s)\n", failed);
    return 0;                                       /* R */
}
```

Test (set `N` to 5 to check by hand): scores 45, 72, 50, 38, 90 give highest 90, lowest 38, average 295 / 5 = 59.00, failed 2 (45 and 38). Note that 50 is a pass, because the test is `< 50`.

Marking guide [15]: array and variables declared [2]; input loop into the array [3]; highest and lowest started from `score[0]` [2]; sum and average as a decimal [2]; correct max and min tests [3]; fail counter [2]; clear output [1].

Engineering lens: swap exam scores for **28-day concrete cube strengths** (MPa) from a pour. The same program gives the maximum, minimum and mean strength, and the count below the **specified characteristic strength** is the number of **non-compliant cubes**, which the site engineer must investigate.

Examiner's tip: never start `highest = 0` and `lowest = 0`. With `lowest = 0` no real score is ever smaller, so the lowest is always reported as 0. Start both at `score[0]`.

Question 3 [25]

3(a) Four types of errors, with examples [2 each = 8]

Like you're 3: a *syntax* error is a spelling mistake; a *run-time* error is tripping over while running; a *logic* error is running very well, but to the wrong house; a *semantic* error is a sentence with good spelling that makes no sense ("the cup drank the milk").

Error	Definition [1]	Example [1]
Syntax	Breaks the grammar rules of C; the compiler refuses to build the program.	`printf("Hello")` — missing `;`
Run-time	The program compiles but crashes or misbehaves while running, usually because of bad data or resources.	`q = v / t;` when the user enters `t = 0` (division by zero); opening a file that does not exist
Logical	The program runs to completion but gives a wrong answer, because the method is wrong.	`avg = a + b / 2;` instead of `avg = (a + b) / 2;`
Semantic	Grammatically correct but meaningless or invalid use of the language, e.g. wrong types or using something that has no value.	`int x = "ten";` (text stored in an int); `printf("%d", 2.5);` (wrong format for a double)

Run-time errors are not just a beginner problem: MIT researchers showed that subtle C errors of this kind (undefined behaviour) occur in real systems and can become security holes (MIT-4).

3(b) Local and global variables, with a program [7]

Like you're 3: a **local** variable is the toy in your own bedroom: only you can play with it, and it is packed away when you leave. A **global** variable is the TV in the lounge: everyone in the house can use it, all day.

- **Local variable** [2] — declared **inside** a function or block; visible only there; created when the function starts and destroyed when it ends; holds garbage unless initialised.
- **Global variable** [2] — declared **outside** all functions, usually at the top of the file; visible to every function below it; lives for the whole program run; starts at 0 automatically.

Program [3]:

```c
#include <stdio.h>

int total = 0;                  /* GLOBAL: outside every function */

void addFlow(int q)
{
    int doubled = q * 2;        /* LOCAL to addFlow only */
    total = total + doubled;    /* a function may use the global */
}

int main(void)
{
    int flow = 5;               /* LOCAL to main only */
```

```
    addFlow(flow);                /* total becomes 10 */
    addFlow(3);                   /* total becomes 16 */
    printf("total = %d\n", total);
    /* printf("%d", doubled);  would NOT compile: doubled is local to addFlow */
    return 0;
}
```

Output: `total = 16`.

Feature	Local	Global
Where declared	Inside a function	Outside all functions
Who can use it	Only that function	Every function in the file
Lifetime	While the function runs	The whole program
Starting value	Garbage unless set	0

Good practice: prefer local variables and pass values as parameters; globals make programs harder to debug.

3(c) Compare and contrast traditional filing and cloud storage [10]

Like you're 3: traditional filing is keeping your drawings in a box under your bed. Cloud storage is keeping them in a magic library you can open from any house, as long as the internet door is open.

Definitions [2]: a **traditional filing system** stores records as paper documents in folders and cabinets (or as separate files on one local computer), organised by a manual index. **Cloud storage** keeps digital files on a provider's remote servers, reached over the internet (STD-1).

Similarities [3]:

1. Both store and organise an organisation's records for later retrieval.
2. Both need a good naming or indexing scheme to find things.
3. Both need access control and a policy on how long records are kept.

Differences [5]:

Point	Traditional filing	Cloud storage
Access	Only where the cabinet is, during office hours	Anywhere, any time, on any connected device
Search and retrieval	Slow, by hand	Seconds, using search
Space and cost	Needs office space, cabinets, paper	No physical space; monthly subscription
Backup and disaster risk	One copy; lost to fire, flood or termites	Automatic copies in several data centres
Sharing	Photocopy or carry the file	Share links; many people edit at once
Dependence	Works without power or internet	Needs internet and a trustworthy provider; raises data-privacy and data-location questions

South African research on choosing cloud providers found **security and data sovereignty** (where the data physically sits, and whose laws apply) among the key selection criteria (RES-1).

Engineering lens: a municipality's **as-built drawings** of water mains were traditionally kept as paper sheets in a plan room. Modern projects keep drawings, models and site records in a **common data environment (CDE)**,

the cloud document system used in BIM, so the designer, contractor and client all work from the same latest revision.

Examiner's tip: for 10 marks, give definitions, at least three similarities and five clear differences. A table of differences is the fastest way to collect marks.

Question 4 [25]

4(a) Five benefits of using the internet [10]

Like you're 3: the internet is a giant road connecting every house in the world, so letters, pictures, lessons and shopping can travel anywhere in seconds.

Name the benefit [1] and explain it with an example [1], five times:

1. **Fast, cheap communication** — email, WhatsApp and video calls reach anyone worldwide in seconds at low cost.
2. **Access to information and research** — search engines, e-journals and open courseware (e.g. MIT OpenCourseWare) put libraries of knowledge on your phone.
3. **E-learning** — online courses, university portals and recorded lectures allow study from anywhere.
4. **E-commerce and online banking** — buying, selling, paying bills and mobile money without travelling.
5. **Collaboration and cloud services** — shared documents, cloud storage and remote work let teams in different towns work together.
6. **E-government and services** — online tax returns, applications and booking.

Context line for an introduction: the internet grew from ARPANET, whose first message was sent from UCLA on 29 October 1969 (UCLA-1).

Engineering lens: the internet lets a utility read **smart water meters** remotely, publish **rainfall and dam-level data** daily, and let consultants share design models with the client the same day.

4(b) Program: PAYE tax from salary [15]

Like you're 3: your salary is a pile of coins sorted into buckets. The first bucket ($100) pays no tax. Coins in the next bucket pay 20c of every dollar, the next 25c, then 30c, then 35c. Each coin only pays the rate of the bucket it sits in.

State your reading: PAYE is a **progressive (marginal)** tax, so each rate applies only to the part of the salary inside its band. The bands are taken as $0–100, $100–300, $300–1000, $1000–2000 and above $2000.

Full tax of each lower band (worked out once, used as constants):

Band	Width	Rate	Tax for the full band	Running total
0 – 100	100	0%	0	0
100 – 300	200	20%	40	40
300 – 1000	700	25%	175	215
1000 – 2000	1000	30%	300	515
above 2000	—	35%	—	—

Algorithm:

```
Step 1: Start
Step 2: Read salary
Step 3: If salary <= 100:       tax = 0
        Else if salary <= 300:  tax = (salary - 100) x 0.20
```

```
       Else if salary <= 1000: tax = 40  + (salary - 300)  x 0.25
       Else if salary <= 2000: tax = 215 + (salary - 1000) x 0.30
       Else:                   tax = 515 + (salary - 2000) x 0.35
Step 4: net = salary - tax
Step 5: Display tax and net pay
Step 6: Stop
```

Flowchart:

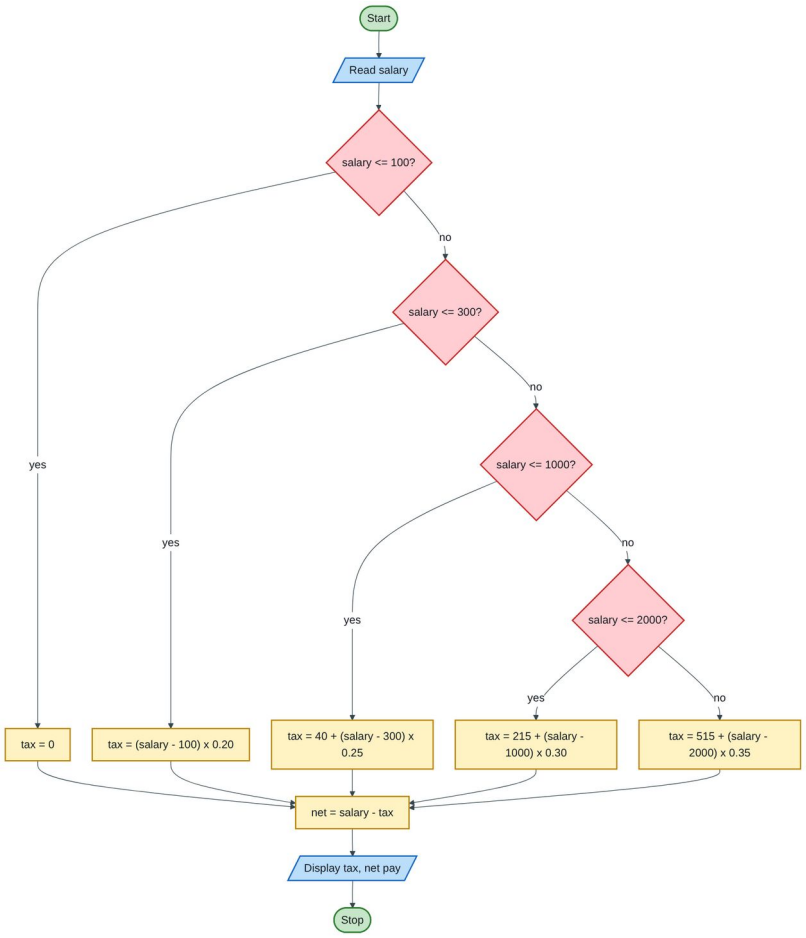


Figure 2.4 — Flowchart

Code (H-D-I-P-O-R):

```c
/* H: PAYE tax using progressive bands 0%, 20%, 25%, 30%, 35% */
#include <stdio.h>

int main(void)
{
    double salary, tax, net;                          /* D */

    printf("Enter monthly salary ($): ");             /* I */
    scanf("%lf", &salary);
    if (salary < 0) {
        printf("Invalid salary\n");
        return 1;
    }

    if (salary <= 100)                                /* P */
        tax = 0.0;
    else if (salary <= 300)
```

```
        tax = (salary - 100) * 0.20;
    else if (salary <= 1000)
        tax = 40 + (salary - 300) * 0.25;
    else if (salary <= 2000)
        tax = 215 + (salary - 1000) * 0.30;
    else
        tax = 515 + (salary - 2000) * 0.35;

    net = salary - tax;

    printf("Salary   : $%9.2f\n", salary);              /* O */
    printf("PAYE tax : $%9.2f\n", tax);
    printf("Net pay  : $%9.2f\n", net);
    return 0;                                           /* R */
}
```

Test table (one salary per band):

Salary	Working	PAYE
$80	first band	**$0.00**
$250	150 × 0.20	**$30.00**
$800	40 + 500 × 0.25	**$165.00**
$1500	215 + 500 × 0.30	**$365.00**
$3000	515 + 1000 × 0.35	**$865.00**

If your lecturer means a flat rate (the whole salary taxed at its bracket's rate), replace the ladder with rates only, e.g. `else if (salary <= 2000) tax = salary * 0.30;`. Then $1500 gives $450. Write one line saying which reading you used.

Marking guide [15]: declarations and input with prompt [3]; validation [1]; correct band ladder with five branches [6]; constants for the lower bands correct [2]; output of tax (and net pay) [2]; layout and comments [1].

Engineering lens: this "banded" calculation has the same shape as Book 1's water **increasing block tariff**. Once you can write one, you can write any piecewise calculation: tariffs, tax, or design tables that change value at set limits.

Examiner's tip: test the boundary values ($100, $300, $1000, $2000). A ladder written with `<` instead of `<=` puts the boundary salary in the wrong band.

Question 5 [25]

5(a) IaaS and PaaS, with examples [4 each = 8]

Like you're 3 (the pizza story): IaaS is renting a kitchen with an oven: you bring the dough and toppings and do the cooking. **PaaS** is a take-and-bake pizza: the base is made for you, you only add your toppings and bake. (SaaS would be ordering a ready pizza.)

Model	Definition [2]	Example [1]	Typical use [1]
IaaS — Infrastructure as a Service	The provider supplies basic computing resources (processing, storage, networks) as virtual machines; the customer installs and manages the operating system, software and data, but not the physical hardware (STD-1).	Amazon EC2, Microsoft Azure Virtual Machines, Google Compute Engine	Hosting a utility's billing server without buying hardware; scaling up at month-end
PaaS — Platform as a Service	The provider supplies a ready platform (operating system, programming languages, database, tools) on which the customer builds and runs its own applications; the customer controls only the application and its data (STD-1).	Google App Engine, Microsoft Azure App Service, Heroku	Developers building and deploying a web app without managing servers

Who manages what:

Layer	On-premises	IaaS	PaaS	SaaS
Application and data	You	You	You	Provider
Operating system and runtime	You	You	Provider	Provider
Servers, storage, network	You	Provider	Provider	Provider

5(b) Five functions of an operating system [5]

Any five, 1 mark each: **process management** (scheduling programs on the CPU); **memory management** (allocating RAM); **file management** (organising and protecting files); **device / I/O management** (drivers for printers, disks, keyboards); **security** (user accounts, passwords, permissions); **user interface** (GUI or command line); **error handling and logging**.

5(c) Program: recursive Fibonacci series [12]

Like you're 3: every new number is the last two numbers added together: 0, 1, 1, 2, 3, 5, 8… The function keeps asking two smaller copies of itself until it reaches 0 and 1, which it already knows.

Rule: fib(0) = 0; fib(1) = 1; fib(n) = fib(n − 1) + fib(n − 2).

Structure chart (how the program is built from modules):

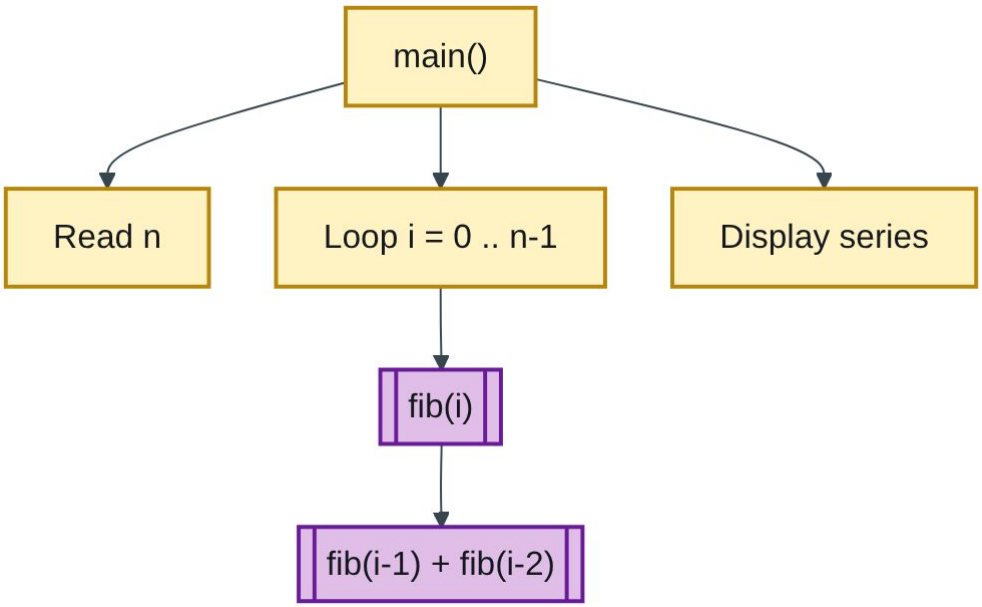


Figure 2.5 — Structure chart

Flowchart of main:

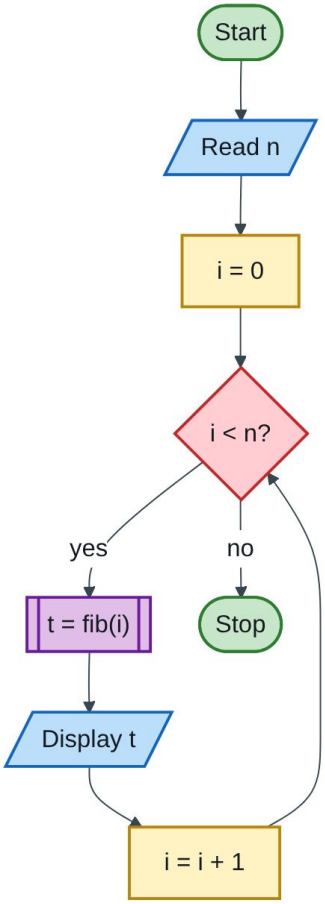


Figure 2.6 — Flowchart

Code (H-D-I-P-O-R):

```c
/* H: Fibonacci series using a recursive function */
#include <stdio.h>

long fib(int n)
{
    if (n == 0) return 0;              /* base case */
    if (n == 1) return 1;              /* base case */
    return fib(n - 1) + fib(n - 2);    /* recursive case */
}

int main(void)
{
    int n, i;                          /* D */

    printf("Number of terms: ");       /* I */
    scanf("%d", &n);
    if (n < 1) {
        printf("Enter a positive number\n");
        return 1;
    }

    printf("Fibonacci series:");       /* P + O */
    for (i = 0; i < n; i++)
        printf(" %ld", fib(i));
    printf("\n");
```

```
    return 0;                          /* R */
}
```

Test: n = 8 prints `0 1 1 2 3 5 8 13`.

Marking guide [12]: header and function definition [2]; both base cases [2]; recursive return [3]; reading and validating n [2]; loop calling `fib(i)` and printing [2]; output layout [1].

Why it gets slow (one line for an extra mark): the naive recursive version recomputes the same values again and again and needs exponential time; MIT's algorithms course fixes this by storing each answer once (memoization), giving about n steps (MIT-2).

Examiner's tip: the paper says "calculate the Fibonacci series", so print the *series* in a loop, not just one term.

Sources for this book

- [STD-1] NIST SP 800-145 — IaaS, PaaS, cloud storage (3c, 5a).
- [RES-1] Bakasa & Pekane, arXiv:2108.09757 — security and data sovereignty in cloud selection (3c).
- [MIT-4] Wang et al., MIT CSAIL — run-time and undefined-behaviour errors (3a).
- [MIT-2] MIT 6.006 Lecture 19 — recursive vs memoized Fibonacci (5c).
- [UCLA-1] UCLA Samueli — first ARPANET message, 1969 (4a).
- Textbook knowledge (tier 4): viruses, CPU components, topologies, OS functions.

Source register

The full register for the solution-book set. Sources marked ✓ are cited in this book; the rest (MIT, UCLA and Wits material used across the set) are further reading on the same topics.

Key	Source	Institution	Used for	In this book
NUST-1	SCS 1101 course outline (uploaded)	NUST	Syllabus, assessment, C as the course language	✓
MIT-1	6.087 Practical Programming in C	MIT	C syntax, pointers, structs, files, coding style	Further reading
MIT-2	6.006 Lecture 19: Dynamic Programming I (Fibonacci)	MIT	Why naive recursive Fibonacci is slow; memoization	✓
MIT-3	Whirlwind Computer, 1944–59 (IEEE Milestone)	MIT	Magnetic-core memory, real-time computing	Further reading
MIT-4	Wang et al., *Undefined Behavior: What Happened to My Code?*	MIT CSAIL	Run-time and undefined-behaviour errors	✓
UCLA-1	First internet message, 29 Oct 1969	UCLA Samueli	History of networks and the internet	✓
UCLA-2	Birthplace of the Internet (IEEE Milestone)	UCLA	ARPANET, packet switching, first node	Further reading
UCLA-3	CS 31 Introduction to Computer Science I, syllabus	UCLA	Intro-programming scope; portability across compilers	Further reading
WITS-1	Sithole (Wits) & Ruhode, *Cloud Computing Adoption: Opportunities and Challenges for SMMEs in South Africa*, arXiv:2108.10079	Wits	Cloud benefits and barriers in Southern Africa	Further reading

Key	Source	Institution	Used for	In this book
WITS-2	*Leadership and the adoption of cloud computing for enterprises in Johannesburg* (2022)	Wits	Drivers and barriers of cloud adoption	Further reading
STD-1	NIST SP 800-145, *The NIST Definition of Cloud Computing*	NIST	SaaS, PaaS, IaaS; 5 characteristics; 4 deployment models	✓
RES-1	Bakasa & Pekane, *Decision Criteria for Cloud Adoption by Large Enterprises in South Africa*, arXiv:2108.09757	UCT	Security and data sovereignty as selection criteria	✓
RES-2	Fu et al., *Error Log Analysis in C Programming Language Courses* (ICCE 2015)	Kyushu University	The errors novice C students actually make	Further reading